11. To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency.

Code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define HISTORY_CAPACITY 5
#define MAX_COMMAND 100

typedef struct {
    char commands[HISTORY_CAPACITY][MAX_COMMAND];
    int count;
} History;

/* Initialize history */
void initializeHistory(History *history) {
    history->count = 0;
}

/* Store command */
void addCommand(History *history, const char *command)

    if (strlen(command) == 0)
        return;

    /* If buffer is full, remove the oldest command */
    if (history->count == HISTORY_CAPACITY) {

        for (int i = 1; i < HISTORY_CAPACITY; i++) {
            strcpy(history->commands[i - 1],
                   history->commands[i]);
        }

        history->count--;
    }

    strcpy(history->commands[history->count], command);
    history->count++;

    printf("Command stored successfully.\n");
}

/* Retrieve a history entry */
void retrieveCommand(History *history, int position) {

    if (position < 1 || position > history->count) {
        printf("Invalid history entry.\n");
        return;
    }

    printf("History %d: %s\n",
           position,
           history->commands[position - 1]);
}
```

```
  GNU nano 8.7.1
/* Retrieve a history entry */
void retrieveCommand(History *history, int position) {

    if (position < 1 || position > history->count) {
        printf("Invalid history entry.\n");
        return;
    }

    printf("History %d: %s\n",
           position,
           history->commands[position - 1]);
}

/* Display complete history */
void displayHistory(History *history) {

    if (history->count == 0) {
        printf("History is empty.\n");
        return;
    }

    printf("\nCommand History:\n");
    printf("-------------------------\n");

    for (int i = 0; i < history->count; i++) {
        printf("%d  %s\n",
               i + 1,
               history->commands[i]);
    }

    printf("-------------------------\n");
}

/* Validate history consistency */
void validateHistory(History *history) {

    if (history->count < 0 ||
        history->count > HISTORY_CAPACITY) {

        printf("History validation failed.\n");
        return;
    }

    for (int i = 0; i < history->count; i++) {

        if (strlen(history->commands[i]) == 0) {
            printf("History validation failed.\n");
            return;
        }
    }

    printf("History validation successful.\n");
```

```
 GNU nano 8.7.1
        history->count > HISTORY_CAPACITY) {

        printf("History validation failed.\n");
        return;
    }

    for (int i = 0; i < history->count; i++) {

        if (strlen(history->commands[i]) == 0) {
            printf("History validation failed.\n");
            return;
        }
    }

    printf("History validation successful.\n");
}

int main() {

    History history;

    initializeHistory(&history);

    printf("Command History Manager\n");
    printf("-----------------------\n");

    /* Store commands */
    addCommand(&history, "ls");
    addCommand(&history, "pwd");
    addCommand(&history, "cd Documents");
    addCommand(&history, "echo Hello");
    addCommand(&history, "gcc program.c");

    displayHistory(&history);

    /* Retrieve an entry */
    printf("\nRetrieving history entry:\n");
    retrieveCommand(&history, 3);

    /* Test capacity limit */
    printf("\nAdding command beyond capacity:\n");
    addCommand(&history, "./program");

    displayHistory(&history);

    /* Validate history */
    printf("\n");
    validateHistory(&history);

    return 0;
}
```

Result:

```
bheeshma@SINGAM: ~/

bheeshma@SINGAM:~/SKILL$ nano Skill11a.c
bheeshma@SINGAM:~/SKILL$ gcc Skill11a.c -o s11a
bheeshma@SINGAM:~/SKILL$ ./s11a
Command History Manager
-----------------------
Command stored successfully.
Command stored successfully.
Command stored successfully.
Command stored successfully.
Command stored successfully.

Command History:
------------------------
1  ls
2  pwd
3  cd Documents
4  echo Hello
5  gcc program.c
------------------------

Retrieving history entry:
History 3: cd Documents

Adding command beyond capacity:
Command stored successfully.

Command History:
------------------------
1  pwd
2  cd Documents
3  echo Hello
4  gcc program.c
5  ./program
------------------------

History validation successful.
bheeshma@SINGAM:~/SKILL$
```

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations.

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_COMMANDS 10
#define MAX_ARGS 20
#define MAX_INPUT 300

/* Structure for a pipeline command */
typedef struct {
    char command[MAX_INPUT];
} PipelineCommand;

/* Store pipeline commands */
int parsePipeline(char *input, PipelineCommand commands[]) {

    int count = 0;

    char *token = strtok(input, "|");

    while (token != NULL && count < MAX_COMMANDS) {

        /* Remove leading spaces */
        while (*token == ' ')
            token++;

        /* Remove trailing newline */
        token[strcspn(token, "\n")] = '\0';

        strcpy(commands[count].command, token);

        count++;

        token = strtok(NULL, "|");
    }

    return count;
}

/* Validate pipeline layout */
int validatePipeline(PipelineCommand commands[], int count) {

    if (count == 0) {
        printf("Error: Empty pipeline.\n");
        return 0;
    }

    for (int i = 0; i < count; i++) {

        if (strlen(commands[i].command) == 0) {
```

```
/* Validate pipeline layout */
int validatePipeline(PipelineCommand commands[], int count) {

    if (count == 0) {
        printf("Error: Empty pipeline.\n");
        return 0;
    }

    for (int i = 0; i < count; i++) {

        if (strlen(commands[i].command) == 0) {

            printf("Error: Empty command in pipeline.\n");
            return 0;
        }
    }

    printf("Pipeline layout is valid.\n");

    return 1;
}

/* Convert command string into arguments */
void parseArguments(char *command, char *args[]) {

    int count = 0;

    char *token = strtok(command, " \t\n");

    while (token != NULL && count < MAX_ARGS - 1) {

        args[count++] = token;

        token = strtok(NULL, " \t\n");
    }

    args[count] = NULL;
}

/* Execute pipeline */
void executePipeline(PipelineCommand commands[], int count) {

    int previousPipe[2];
    int currentPipe[2];

    pid_t pids[MAX_COMMANDS];

    for (int i = 0; i < count; i++) {

        /* Create pipe between commands */
        if (i < count - 1) {
```

```
  GNU nano 8.7.1
/* Execute pipeline */
void executePipeline(PipelineCommand commands[], int count) {

    int previousPipe[2];
    int currentPipe[2];

    pid_t pids[MAX_COMMANDS];

    for (int i = 0; i < count; i++) {

        /* Create pipe between commands */
        if (i < count - 1) {

            if (pipe(currentPipe) == -1) {
                perror("pipe");
                return;
            }
        }

        pids[i] = fork();

        if (pids[i] < 0) {
            perror("fork");
            return;
        }

        /* Child process */
        if (pids[i] == 0) {

            char *args[MAX_ARGS];

            /* Connect previous command's output
               to current command's input */
            if (i > 0) {

                dup2(previousPipe[0], STDIN_FILENO);

                close(previousPipe[0]);
                close(previousPipe[1]);
            }

            /* Connect current command's output
               to next command's input */
            if (i < count - 1) {

                dup2(currentPipe[1], STDOUT_FILENO);

                close(currentPipe[0]);
                close(currentPipe[1]);
            }

            /* Convert command into arguments */
```

```
  GNU nano 8.7.1
            /* Convert command into arguments */
            parseArguments(commands[i].command, args);

            /* Execute command */
            execvp(args[0], args);

            /* If execvp fails */
            perror("execvp");
            exit(1);
        }

        /* Parent process */

        if (i > 0) {

            close(previousPipe[0]);
            close(previousPipe[1]);
        }

        if (i < count - 1) {

            previousPipe[0] = currentPipe[0];
            previousPipe[1] = currentPipe[1];
        }
    }

    /* Wait for all processes */
    for (int i = 0; i < count; i++) {
        waitpid(pids[i], NULL, 0);
    }
}

int main() {

    char input[MAX_INPUT];

    PipelineCommand commands[MAX_COMMANDS];

    printf("Pipeline Execution System\n");
    printf("-------------------------\n");

    printf("Enter pipeline command:\n");
    printf("Example: ls | grep txt | wc -l\n\n");

    printf("pipeline> ");

    fgets(input, sizeof(input), stdin);

    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';

    /* Parse pipeline */
```

```c
    PipelineCommand commands[MAX_COMMANDS];

    printf("Pipeline Execution System\n");
    printf("-------------------------\n");

    printf("Enter pipeline command:\n");
    printf("Example: ls | grep txt | wc -l\n\n");

    printf("pipeline> ");

    fgets(input, sizeof(input), stdin);

    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';

    /* Parse pipeline */
    int count = parsePipeline(input, commands);

    printf("\nCommands in pipeline: %d\n", count);

    /* Display execution order */
    printf("\nExecution Order:\n");

    for (int i = 0; i < count; i++) {
        printf("%d. %s\n",
               i + 1,
               commands[i].command);
    }

    /* Validate pipeline */
    if (!validatePipeline(commands, count)) {
        return 1;
    }

    printf("\nExecuting pipeline...\n\n");

    /* Execute pipeline */
    executePipeline(commands, count);

    printf("\nPipeline execution completed.\n");

    return 0;
}
```

Result:

```
bheeshma@SINGAM: ~/

bheeshma@SINGAM:~/SKILL$ nano Skill11b.c
bheeshma@SINGAM:~/SKILL$ gcc Skill11b.c -o s11b
bheeshma@SINGAM:~/SKILL$ ./s11b
Pipeline Execution System
-------------------------
Enter pipeline command:
Example: ls | grep txt | wc -l

pipeline> ls | grep txt | wc -l

Commands in pipeline: 3

Execution Order:
1. ls
2. grep txt
3. wc -l
Pipeline layout is valid.

Executing pipeline...

0

Pipeline execution completed.
bheeshma@SINGAM:~/SKILL$
```